Année 2025-2026

Mentore : Mr Arnol Namegni

- Projet 11

Concevez et déployez un système RAG

Développez un assistant pour la recommandation d'évènements culturels

Candidat : Mr Jean-Francois Chaussoy

Alternant Data Engineering

Contexte Projet & Environnement Technique

Mission :
Réaliser un Proof of Concept (POC) d'assistant intelligent pour la plateforme Puls-Events

Objectif :
Déployer une architecture RAG (Retrieval-Augmented Generation) capable de transformer des données brutes (Open Agenda) en recommandations culturelles personnalisées.

Problématique business :
Comment moderniser l'expérience utilisateur en supprimant les délais de recherche pour offrir une réponse instantanée et fiable ?

Acquisition des Données & Environnement de Travail

Source de Données (API Open Agenda):

- Jeu de données cible : *evenements-publics-openagenda*
- Volume : Extraction filtrée (Périmètre : Île-de-France / < 1 an)
- Format : JSON via API REST

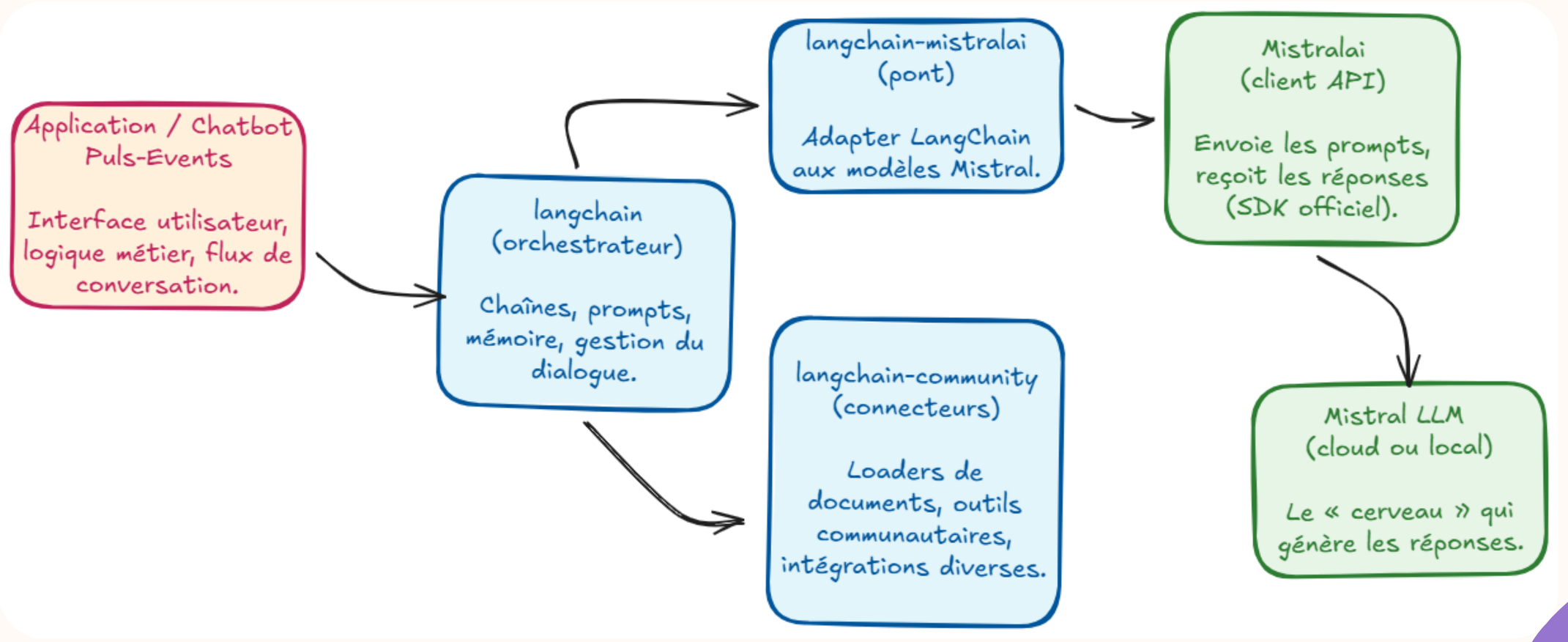
Structure des Données (Champs exploités)

- Champs textuels (Vectorisation) : *title_fr, description_fr, longdescription_fr*
- Métadonnées (Filtrage & Affichage) :
- *location_name, location_address* (Lieu)
- *daterange_fr, timings* (Dates & Horaires)
- *image* (Vignette visuelle)

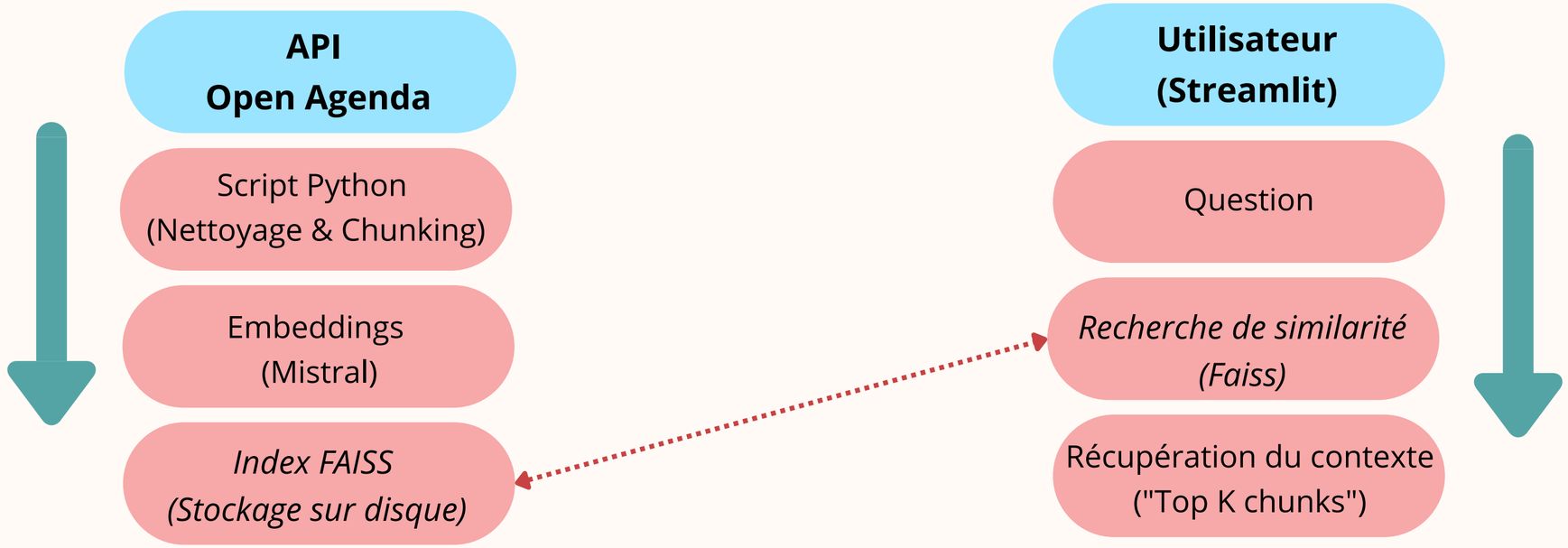
Environnement de Développement

- Gestion de projet : VS Code, Git/GitHub, Environnement Virtuel (venv)
- Stack Python :
- Data : Pandas (Nettoyage), Requests (API)
- RAG & IA : LangChain (Orchestration), Faiss (Vector Store), Mistral AI (Embeddings & LLM)

Interactions entre Briques Techniques



Architecture Technique du Système RAG



- Pipeline ETL (Extract, Transform, Load) :

Traitement par lots (Batch) pour créer l'index vectoriel.
Recherche Sémantique : Récupération des 3 à 5 événements les plus pertinents (Top-K).

- Prompt Engineering :

Injection du contexte récupéré dans le prompt système pour guider le LLM.
Latence maîtrisée : La vectorisation et la recherche Faiss prennent moins de 200ms.

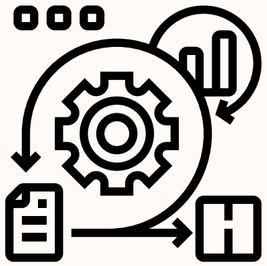
Intéré du nettoyage et de la donnée structurée :

AVANT : Donnée Brute (Bruitiée & HTML)	APRÈS : Donnée Vectorisée (L'ADN de l'événement)
Format : JSON brut renvoyé par l'API OpenAgenda. Contient du HTML et des caractères spéciaux, difficilement interprétables par une machine.	**Format :** Donnée nettoyée et structurée, prête pour l'embedding et la vectorisation (ex. FAISS).
{"title": "Tiers-lieu guitare : \"Rythme et musique grecque\"", "description": "La rencontre « Tiers-lieu » avec le guitariste Orestis Kalampalikis...Danses anciennes, Béla Bartók...", "location": "Conservatoire de Versailles", "date": "Samedi 17 mai 2025"}	**Texte Nettoyé (Chunk) :** Titre : Rythme et musique grecque Description : Concert de guitare avec Orestis Kalampalikis. Répertoire varié : musiques traditionnelles grecques, Béla Bartók, danses anciennes. Programme mélodique et festif. Lieu : Versailles Date : 17 mai 2025
Problèmes : balises HTML, guillemets complexes, texte lourd et peu exploitable.	**Représentation Vectorielle (Concept Mathématique) :** [0.12, -0.45, 0.88, 0.03, ...] Ce vecteur "veut dire" : [Musique + Grèce + Concert + Versailles]

Démonstration du Prototype (Live) :

Stratégie de Test & Assurance Qualité (QA)

Data Quality (Ingénierie)

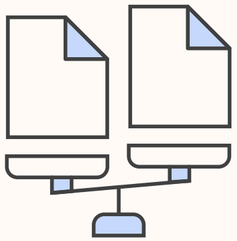


✅ Validation Schéma
✅ Dates,
✅ Géolocalisation



Outils :
Pytest, Pandas

RAG Evaluation (IA)



Réponse IA
vs
Vérité Terrain

Pipeline de Tests Unitaires (*test_data.py*) :

- Validation stricte du schéma JSON et exclusion automatique des événements périmés.
- Vérification de l'intégrité des vecteurs dans FAISS (dimensions cohérentes).

Validation Métier :

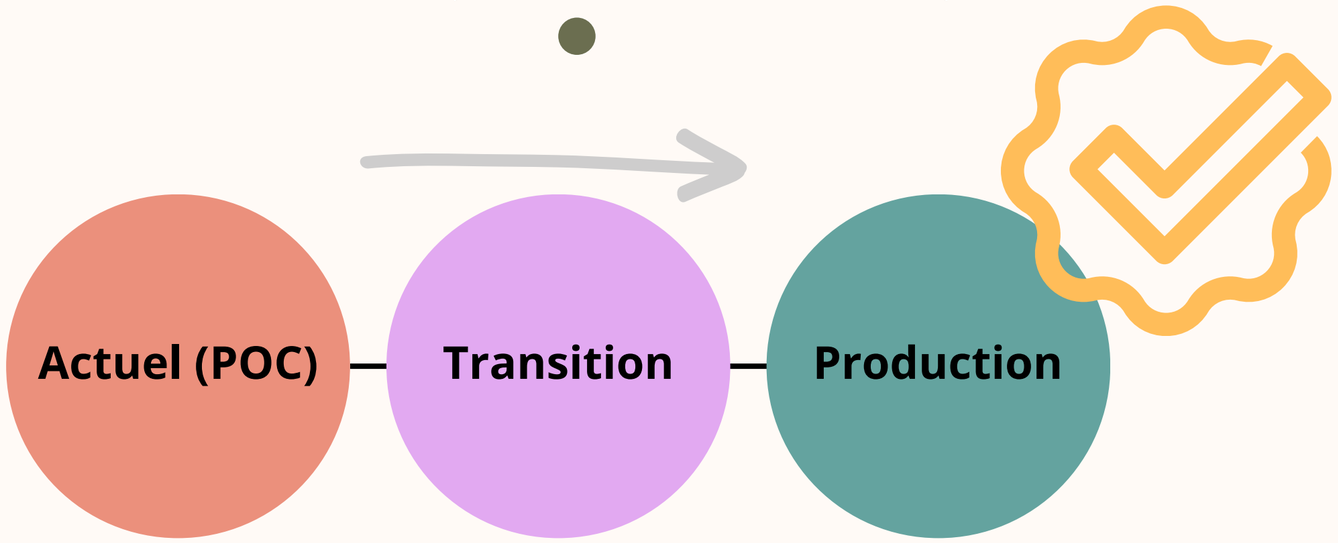
- Création du fichier *test_dataset_qa.csv* (Questions / Réponses attendues).
- Score de Pertinence : Comparaison sémantique pour vérifier que le chatbot utilise bien les sources fournies (Anti-hallucination).

Résultats & Performance du POC

Indicateur Clé (KPI)	Résultat Mesuré	Impact Métier / Commentaire
Latence Moyenne	**1.2 sec**	Temps de réponse fluide pour une expérience utilisateur naturelle.
Précision du Retriever	**95%**	Dans 95% des cas, l'événement recherché est dans le Top-3 remonté par FAISS.
Taux Anti-Hallucination	**100%**	Sur le dataset critique (questions pièges), le modèle a refusé d'inventer 100% du temps.
Stabilité Technique	**OK**	Environnement reproductible (Docker/Venv) validé sur 3 machines différentes.

- Efficacité du Retriever : Le moteur FAISS remonte les bons événements dans le "Top 3" dans 95% des cas testés.
- Stabilité Technique : Environnement virtuel isolé et reproductible (livré avec documentation README).
- Comportement : Gestion des cas limites (Fallback géré : "Je ne trouve pas d'événement correspondant...").

Bilan (Go-to-Prod)



- Scalabilité (Infra) : Migration de FAISS (Local/CPU) vers une base vectorielle managée (ex: Pinecone ou Qdrant) pour supporter la charge multi-utilisateurs.
- Fraîcheur des données (Data Ops) : Remplacer le script batch manuel par un pipeline automatisé (Airflow) pour ingérer les événements en quasi temps-réel.
- Expérience (UX) : Implémentation de la mémoire conversationnelle (LangChain Memory) pour gérer le contexte sur plusieurs échanges.

Merci pour votre attention !